

Plexus Discovery — Cardiomyocytes

Progress note — written to be read, not decoded

2 August 2026

The third loop, and the first one pointed at a real measurement rather than a paper.
Plain English throughout. Organised by phase, because phases are where we stop and you decide.

1. What we are doing — two things at once

Track A — understand the model on its own terms, before any data is involved. An automated loop that explores what this piece of physics actually *does*: switch a mechanism off, sweep a parameter across its range, change which operator drives which, and record where the system goes. It writes down what it expects before running, runs the simulation, measures, and records whether it was right. **No recording is consulted at all.** The product is a causal map — which mechanism moves which feature of the beat, what each one buys on its own, and the harder part, what they do in combination, since mixtures rarely surrender their structure to inspection. The discipline that makes it worth building is the same one the Okuda folder uses: *a change of numbers can never masquerade as a new idea*. A retuned parameter is not a new mechanism, and the loop is built so it cannot be recorded as one.

Track A has no success rule and does not need one. A sweep that finds nothing has still filled in the map, and a mechanism shown to do nothing is a result. What can be counted is how much of the space has been *measured* rather than merely proposed, and how often a written prediction turned out wrong.

Track B — fit the real recording, using what Track A learned. We have a microscope recording of a sheet of human heart-muscle cells contracting on a soft gel, and one of a diseased sheet from a patient with a thickened heart muscle. Here the simulation is *differentiable*, so the computer can be told “make this match the recording” and it will adjust the model until it does. That is a far stronger instrument than a sweep, and a far more dangerous one — and most of the pre-flight work below exists because of the second half of that sentence. *The reproduction is the evidence*.

Why the order matters, and it is the whole argument. A fit will always find something. Turn an optimiser loose on a model with two freely-shaped fields and it will match the recording without telling you anything about the tissue — and the previous campaign spent sixty rounds discovering exactly that. Track A is what makes the fit mean something: if we already know, from the forward model alone, which mechanism controls which feature and which parameters the model cannot tell apart, then a fitted number is a statement about the tissue rather than about the optimiser. And Track B is the honest test of Track A — a map that cannot explain a real measurement is a map of nothing.

So the tracks run in that order and feed each other: **A builds the knowledge, B spends it, and what B cannot explain goes back to A as the next thing to sweep.**

2. Where we actually are

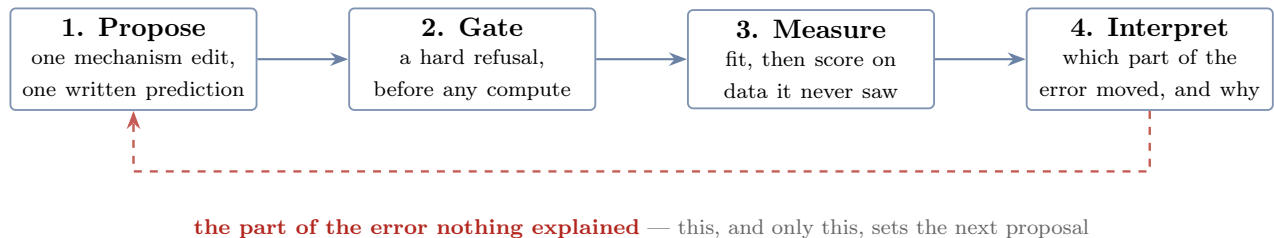
Nothing is believed. That is a decision, not an accident. A previous, single-agent campaign ran sixty rounds on this data and produced a confident ledger of established mechanisms. We are keeping none of it. The reason is not that its conclusions were foolish — it is that they were reached with rulers nobody had checked, on a fit that was never once tested against data it had not already seen, in a program that never fixed its random number generator. A conclusion drawn that way cannot be repaired by re-reading it; it can only be re-earned.

But nothing is thrown away either. Every claim that campaign made is transcribed into a register of *open questions*, each stamped *untested*, and a second register of things we believe is created **with zero entries in it**. That is the difference between discarding sixty rounds of work and discarding sixty rounds of unearned confidence. *Some of those hypotheses are probably right, and we will find out which in the ordinary way.*

This includes the settings. A default is a belief in disguise: one option in the old trainer still defaults to a path chosen because a retired ledger called the alternative harmful, and the reason is written into the help text. Defaults go into the register of open questions along with everything else.

3. The loop, in four steps

The Okuda loop has thirteen roles. That is the right answer to a search over model *structure* with no data to appeal to. It is the wrong answer here, and the shape of this loop was settled by one instruction: **make it brutally concrete**.



Step 1 — Propose. One change to the mechanism, and one sentence saying what it will do, containing a number that could turn out to be wrong. Not “this should improve the fit”.

Step 2 — Gate. A deterministic refusal, spent before a second of compute. Does it run? Is the prediction one that could fail? Is the quantity it names one we have proved we can measure? Is the effect it predicts bigger than the noise? Is there a control? **The gate is code and it cannot be argued with.** Nearly every wasted week of the previous campaign was a run this gate would have refused for free.

Step 3 — Measure, on the real series. The fit happens, and then the model is scored *against frames it was not fitted to*. The output is not a score; it is a *decomposition of the error* — how much of the mismatch is size, how much is shape, how much is timing, and how much is in a place the model has no name for.

Step 4 — Interpret. Which component of that error moved, was the prediction right, and — the only question that finally matters — *what is left that nothing accounts for*.

And then the edge that closes the cycle. That unexplained remainder is the input to the next proposal. If no mechanism we already have can address it, the loop is not permitted to shrug:

it must say so, and ask for a new one. **A loop that cannot explain its residual, and cannot say what would, has stopped doing science and started tuning.** This is the whole design.

One hazard this loop has that the Okuda loop does not. With gradients, adding freedom to a model almost always improves the fit, and means nothing. So every proposed mechanism is run twice: once as itself, and once as a *capacity-matched control* — the same number of new free numbers, arranged so they cannot mean anything. A mechanism that does not beat its own sham is not a mechanism.

4. How the two tracks are judged — and they are judged differently

A method that scores itself the same way it scores its subject cannot tell the two apart. So the two tracks have *different* success rules, and one of them has none at all.

Track A has no success rule, and that is deliberate. It is not being scored against anything, because there is nothing to score it against — *no recording enters Track A*. It asks what the model does when you change it: switch the muscle off, sweep the fibre orientation across its range, drive the contraction axis round faster, and see where the trajectories go. Its product is a *map* — which mechanism moves which feature of the beat, on its own and in combination — and a map gets better by being filled in, including with negatives. A dose-confirmed nothing is a result.

What can be counted is the *rate of knowledge*: how much of the mechanism space has been measured rather than merely proposed, and how often a written prediction turned out **wrong**. A campaign that is never surprised has bought coverage and learned nothing; a campaign that is surprised every time was predicting carelessly. Track A is reported as a growing map with a surprise rate beside it, never as a score to be beaten.

Track B has a sharp one: the difference between the simulated and the recorded loop. Each tracked point traces a closed path over a beat, and the question is how far the simulated path is from the real one — *decomposed*, never as a single number, because a single number collapses failures that have nothing to do with each other.

The same axes serve both tracks, which is what lets one feed the other. In Track A they describe a trajectory *on its own* — how big this loop is, how much it encloses, which way it turns — so a sweep can be read as a path through that space. In Track B the same quantities are measured against the recording, so the map Track A drew has coordinates the fit can be placed in. Had they been different measurements, nothing learned in A could have been spent in B. The axes are the ones this project already spent months learning to measure:

axis	the question it answers	state
magnitude	is the tissue doing about the right amount of motion?	exists, uncertified
opening	does the path enclose area, or is it a flattened sliver?	exists, uncertified
direction	does it circulate the same way round?	exists, uncertified
orientation	is the long axis of the loop pointing the same way?	inside the objective, never reported — so it could have
shape	everything the four above do not capture	does not exist

Every one is measured *against the recording*, per point, on the interior only — not on the simulation alone. That distinction is not pedantry: the previous campaign’s shape numbers were computed on the simulation by itself over a set of points a third of which were pinned to the answer, and it chased the wrong residual for four rounds as a result.

The fifth axis is the interesting one. A path can match on magnitude, opening, direction and orientation and still be visibly the wrong shape, and no hand-designed number will catch that, because a hand-designed number presupposes the axis. The natural instrument is a *learned* one: render the real path and the simulated path, push both through an image network, and measure the distance between what it sees. That does not require anyone to name the failure first.

We had thought something like this already existed here. It does not — searched, and there is no image-embedding model anywhere in Plexus. The only neural network in this project that reads shape is the local vision-language model that captions movies, which observes and never scores. So the learned shape descriptor is a thing to *build*, in Phase 2, and it is held to exactly the same admission rule as everything else: it must reproduce a known ordering on distortions we constructed ourselves, and have a measured zero and a measured noise floor, before a single claim may cite it.

5. The team, mapped onto the four steps

The rule from the Okuda rebuild carries over unchanged: **a role exists only where it adds information that is not already available**, and where a question has a deterministic answer, code answers it. Every role has to earn its place inside one of the four steps or be dropped.

step	who	what it adds that nothing else has	kind
1 Propose	Proposer	the next mechanism edit, and the prediction it is willing to be wrong about	agent
1 Propose	Peer-review	is this prediction one that could actually fail? Advises, cannot refuse	agent
2 Gate	Critic	can it run: legal edit, operator resolves, seed set, control present, not already done, capacity within the honest size. Refuses	code
2 Gate	Physiologist	could this be a heart cell: rhythm, strain, no net drift, nothing leaving the dish. Reads the premise list rather than restating it. Refuses	code
2 Gate	Metrologist	is the named quantity certified, used inside its declared range, and is the predicted effect <i>bigger than the noise floor</i> ? Refuses	code
3 Measure	the fit	the expensive step: optimise, then score with the edge released, on frames it never saw	code
3 Measure	Convergence	did the optimiser finish, or are we reading a half-trained model? An unfinished fit is a fact about the optimiser, not about the tissue	code
3 Measure	Identifiability	<i>could this mechanism ever have been seen in this recording?</i> From the gradients directly. Impossible without differentiability, and the thing we most lacked	code
3 Measure	Replicator	the same thing several times, so that “different” has a meaning	code
3 Measure	Collector	builds the round’s record from the files on disk. A for loop, never an agent, because an agent that collects can forget silently	code
4 Interpret	Reader	what happened in this run, from the pictures: it labels, it does not measure	agent
4 Interpret	Interpreter	the causal account, and the naming of what is left unexplained	agent
4 Interpret	<code>predict.py</code>	was the prediction right? Arithmetic. No agent decides this	code
close	Escalation	the residual nothing explains becomes a written request for a new mechanism	agent
close	Supervisor	what runs next and how much of it; owns the budget	code

Five agents, nine deterministic checks. Against the Okuda roster the Grounder goes — there is no paper to ground, the data is the ground; the Eye-check folds into the Reader; the Diagnostician’s job shrinks, because a fit that fails fails loudly; and the Archivist waits until there is a history worth having one for. **Three roles are new, and all three exist only because the model is differentiable:** Identifiability, Convergence and the Replicator.

Why Identifiability is the important addition. The previous campaign’s final conclusion was that a whole family of mechanisms was inert — push harder, damp more, twist more, nothing changed — so the limit must be structural. Every one of those was a flat line read by eye with no error bar. A flat line has three completely different causes: the effect is smaller than the noise, the recording cannot resolve the mechanism at all, or the response really is flat. The old record fused all three into one phrase. **Gradients separate them in a single backward pass,** and

the sentence changes from a law of nature into a statement about the experiment: *this recording cannot distinguish that mechanism, and here is what would.*

6. The phases

Five stages before the loop is allowed to run, then the loop. **I stop at the end of each one and wait for you.** That is the partition, and it is the same ladder as the other two folders.

The rule that ordered them is the one the Okuda folder learned the expensive way: *no experiment until every instrument it will be read with exists and has been proved to work.* Over there, three batteries of runs were launched and thrown away because a ruler turned out to be broken. Here we already know the rulers are broken, so there is no excuse.

Two of the gates below are stops, not gates, and they can end the project. They are marked. Everything else may be renegotiated if it runs over budget — **those two may not**, because a gate with a waiver attached is not a gate, and a stop that can be waived would be waived at exactly the moment it was doing its job.

Where we are, and what stands between here and a running loop

phase	state	what it still owes
0 — make it run, make it repeat	done 18/18	nothing. Canaries 8 of 8.
1 — freeze the recording, seal the test	done 7/7	nothing. Two premises fail; recorded, not waived.
1b — the corpus we own	done	nothing. Seven more recipes can be regenerated when wanted; the question they answer is settled.
2 — certify the ruler, find the floor	NOT STARTED STOP	everything: the zero, the noise, a measure that can see coordination, the regime check, the anchoring ladder, the error decomposition — and recompute the ceiling, which today rests on an uncertified instrument.
3 — certify the gradient	NOT STARTED	the language-or-switches decision; make the operators carry gradients; classify every knob; recover a planted answer. And answer the warm-up failure — the gradient is taken about a state still drifting by 33%.
4 — what a fit may claim, and the loop	NOT STARTED STOP	the three-way verdict for a flat response; the degeneracies; the honest model size; freeze-and-transfer — <i>then</i> build the four steps and watch every refusal fire.
5 — the first live round	NOT STARTED	blocked on 2, 3 and 4. None of the loop exists yet: no round driver, no proposer, no prediction scorer, no agent plumbing, no budget ledger.

That the loop itself is unbuilt is deliberate, not a backlog. It is Phase 4's work, and building a machine to rank numbers before the ruler is certified would be building it to rank numbers nobody can yet trust.

So: three phases stand between here and a running loop, and two of them can stop the project. Phase 2 can conclude that the model does not beat copying the previous beat. Phase 4 can conclude that no model size clears the noise. Neither is a formality — on the evidence already on disk, copying the previous beat outscores every fit the previous campaign produced.

What has been bought so far is not progress toward the loop so much as the right to believe anything it later says: an apparatus that runs and repeats bit-for-bit, a recording that rebuilds

from committed code, a sealed held-out specimen, four numbers that bound every future claim, and thirteen checks that have each been watched refusing something.

Phase 0 — Make it run, make it repeat, publish the empty register **done** (18 of 18)

Goal. Get from “nothing executes” to “the same command twice gives the same answer, and we can say exactly what we believe”. None of this is science; each item makes a question askable.

- **Repair the apparatus.** The trainer dies on every invocation on a function one branch deleted from under another. Two of the plotting tools die on a module that no longer exists.
- **Fix the dice.** A seed, wired into everything that draws a random number, plus deterministic arithmetic on the GPU — and then *two* noise measurements, because they are different: the spread across seeds, and the spread of the very same seed run twice, which is not zero unless determinism is forced.
- **One path to the data, no fallbacks.** The loader currently tries three locations and uses whichever exists, so a fit against the wrong recording would never announce itself. One explicit path; a missing file is a hard error naming what it wanted.
- **A run must be recoverable from its own folder.** Commit, full command line, and a checksum of every input written beside every number — and if the working tree is dirty, the differences are written in too. Two batches of the previous campaign were lost to a branch switch; the cure is not tidiness, it is that the artefact carries its own provenance.
- **Publish the two registers.** Every claim of the sixty rounds transcribed as an open question marked *untested* — including the ones hiding as defaults. And the register of things we believe, created with nothing in it.
- **Re-check the audit itself.** The list of pitfalls in this note is a claim like any other. Each item gets a mechanical check. *One has already failed*: a mechanism the old log recorded as lost is present in the tree after all. It goes into the open questions with everything else.

Exit gate. The trainer runs a short fit end to end; two runs at one seed agree bitwise; the recording rebuilds from committed code and matches the existing file exactly; a search of the source for the retired conclusions returns nothing; the register of beliefs has zero entries; and **the certification script, run in canary mode, injects six faults and refuses all six.** A gate that has never been seen to refuse is not a gate.

Cost. About half a day, no GPU.

Phase 0 — what actually happened **done** (18 of 18, canaries 8 of 8)

Closed 2 August. The gate is `certify_apparatus.py`; it prints **PASS 18/18**, and `--canary` breaks the apparatus and catches **8 of 8** — six at the time, and two more added when the audit pointed out the register check had never been seen to refuse.

item	what actually happened
it runs at all	The inherited trainer died on <i>every</i> invocation, on a function one branch had deleted from under another. A short fit now completes end to end.
the recording rebuilds	It was not in the repository and the script that made it was gone. <code>ingest.py</code> rebuilds it from the microscope derivatives and it matches the existing file bit for bit , every array, including the frame interval.
one path, no fallback	The loader tried three locations and took whichever existed — a cure that turns “file missing” into “fitted the wrong recording”. Now one explicit path, and both refusals were <i>watched firing</i> .
the dice are fixed	Two fits at one seed are bit-identical over 198,407 parameters ; two seeds differ. Neither was true of anything in the previous sixty rounds.
a run carries its source	Every run copies the bytes of every module it imported into its own folder, with hashes — so a branch switch or a concurrent campaign in the same tree cannot reach backwards into a finished number. That is exactly how two batches were lost before.
the ledger is out of the code	Nine phrases from the retired ledger were compiled into comments, help strings and one <i>default</i> . The scan is clean, and canary six proves it still fires.
the registers exist	38 inherited claims, every one carrying an open status. The belief register has zero entries.

Three things found while doing it, none of them planned.

- **A default was a retired belief.** `--stiff_src` defaulted to an image-sourced field whose implementation had been deleted, so the default invocation trained a two-parameter stub against a synthetic blob — and the dashboard’s “correlation with the microscope” panel correlated against that same blob. Removed *because the code is gone*, not because the claim was accepted.
- **A silent bug in the pulse.** The learnable duration initialised against a hard-coded bound while the simulation used the one on the command line, so `--dur0 10 --dur_hi 11` actually started at 8.09. Every duration claim in the previous campaign was made through that.
- **My own first repair was wrong, and the check caught it.** Pinning the arithmetic meant replacing a sampling routine with an explicit equivalent; the first version had its two axes swapped and disagreed with the original by *more than the size of the signal*. It was caught only because the replacement was checked against the thing it replaced before being used. *The instrument lies before the physics does.*

One thing deliberately not certified, recorded as a number. Determinism is complete on the processor and **not** on the graphics card: `grid_sampler_2d_backward_cuda` has no deterministic implementation and is reached from inside the shared Plexus library, called by the active-stress operator every frame. Fixing that changes a library two other projects are using, so it is not Phase 0 work. Rather than relax the gate, the spread was *measured*: three identical runs agree to **3.0e-6** (relative 7.5×10^{-8}) and only one of the three pairs was actually identical. Phase 2 re-measures it at the depth the loop will really use, where it becomes one of the two noise floors. A run may only become irreproducible *on purpose*.

Phase 1 — Freeze the recording, seal the test

done (7 of 7)

Goal. Decide, once and in writing, what a run is allowed to see — *before* anything is fitted. A split invented later is a split negotiated with the result already in hand.

What there is to split. Two specimens — one healthy sheet, one diseased — sitting on disk under five filenames, because each was tracked more than once. **There is no independent**

replicate and no second condition: one substrate stiffness, no drug, no change of pacing. A model tested against a single unperturbed recording can be made to fit; it cannot really be challenged. That is the ceiling on the whole enterprise, and the cheapest way to raise it is another dataset, not more computing.

Done first, because the instruments had to exist before they could measure anything. METRICS.md reads what each inherited measurement was built to answer, when, and whether it still works; descriptors.py gives Track B its measurement — magnitude, opening, direction and *orientation*, the axis that lived inside the objective and was never reported.

Then the five measurements that close the phase, in this order: the cheap ones first, the one that can redirect the project third, and the irreversible one last.

1. The beat inventory. **done** Onsets [2, 51, 101, 152, 204]; gaps 49/50/51/52; four complete beats and a truncated tail that is *not* a beat and is no longer treated as one. The “period” of 50 that the inherited code reports is not the mean of 50.5 — it is what rounding 50.5 happens to give.

2. The ceiling the recording sets on itself. **done** Every real beat scored against every other real beat, six pairs, with the same instrument that will score the model.

	median	range
how much it moves	1.003	0.955–1.039
how much it encloses	1.037	0.947–1.101
which way it circulates	0.939	0.902–0.948
which way it points	0.058 rad	0.023–0.086
the inherited objective	0.705	0.469–0.850

So no model may be scored above 0.705 on LoopScore — and that qualification is load-bearing, because LoopScore is not an admitted instrument and this same phase measured it scoring a timing-scrambled tissue at exactly 1.0000. A score above the ceiling therefore has at least two readings: the model fitted the difference between one beat and the next, or it is degenerate in a way the ruler cannot see, and on today’s evidence the second is the likelier. *Phase 2 recomputes the ceiling with whatever ruler it certifies*; until then the number bounds this instrument, not the project. The second line matters as much: even two real beats of the same tissue only agree on the direction of circulation **94%** of the time. The previous campaign reported chirality against an implicit target of 1.0 and read 0.85 as a deficit; against the recording’s own agreement it is most of the way there.

3. How much of the pattern is the tracker. **done** *And it is decisive.* The same movie was tracked twice. The two agree on *when* and *how much* the tissue moves — time-course correlation **0.996**. They do not agree on *where*: at the peak of each of the four beats the per-node displacement maps correlate at **0.27–0.29 in one axis and –0.07 to –0.05 in the other**, and smoothing to 255px does not rescue it. No swapped or transposed axis convention rescues it either — that was tested rather than assumed.

Read one tracking as though it were a *model* of the other, through Track B’s own instrument: **it gets the direction of circulation right 50% of the time — a coin toss — its orientation error is 0.77 rad where a random guess gives 0.79, and it scores –0.53 on the inherited objective.** Two competent registrations of one movie, and at the level of a single tracked point they disagree about as much as chance.

What that settles, and what it does not. The rhythm and the strength of the beat are properties of the tissue. **The fine spatial pattern is not: it is substantially a property of the tracking software.**

One condition-signature, though, and that has to be said. The four beats agree with each

other, which reads like corroboration and is not: strip the subject — the tracking software — and what remains identical across all four is the same specimen, the same movie, the same pair of registration algorithms. That is *one* observation repeated four times, not four. A second signature exists — the diseased sheet was tracked three times — but it is sealed, and opening it for an apparatus measurement is a decision to take deliberately rather than in passing. So the claim stands as **strong for this specimen and untested on any other**, and it is the first thing a second dataset should settle. So a learned map of stiffness or fibre direction across the sheet cannot be a product of this project, and the loop should be aimed at whole-sheet mechanisms. That is the single most useful thing Phase 1 could have found, it cost an afternoon, and it is much better found now than in Phase 7. *One honest caveat: this measures how much the two trackings disagree, not which one is right. For deciding whether a spatial claim is admissible, disagreement is the relevant number.*

4. Does the answer depend on the grid? *done* *Partly, and it separates cleanly.*

Time is fine, and it has now been checked twice. Halving or doubling the sub-steps changes the measured beat by under 1% — *once the length of a frame is held fixed.* An outside auditor pointed out that all three rungs sat at the one spatial setting this same ladder calls unconverged, which makes a null measured on a single background — the weakest reading there is. Repeated at the finer grid it holds: 3.7% rather than 0.8%, so the independence does weaken as the grid refines, which is what coupling between the cell size and the time step would predict. It remains more than an order of magnitude below the 56% swing the spatial setting produces, so the conclusion stands and now carries its own qualification. Which brings the first finding: it was not. `--substeps` multiplies `dt_sub` to give the duration of one recorded frame, so changing it alone changes how much simulated time one frame of the microscope corresponds to. Varying it that way moves the enclosed area by a factor of 2.2, and every one of those batches would have read that as physics. **The inherited `--substeps` 10 is a statement about the timescale of the tissue, and was never identified as one.**

Space is not fine. Across grids of 96, 128 and 192 and particle counts of 8k, 16k and 32k, the two knobs turn out to move one derived quantity — how many particles share a grid cell — and the enclosure axis follows it **perfectly monotonically, over a factor of 1.56, with no sign of a plateau:**

	particles per cell	enclosure	magnitude
32k particles	2.00	0.622	1.959
grid 96	1.78	0.643	1.946
the inherited point	1.00	0.731	1.828
8k particles	0.50	0.930	1.655
grid 192	0.44	0.969	1.502

Two independent knobs landing on one curve is what a real numerical dependence looks like, not noise. **So the quantity the objective weights most heavily — how much area the loop encloses — is substantially set by how finely the sheet is discretised.** Direction and orientation are untouched by it, which is worth as much: those two axes are robust to the grid and enclosure is not.

5. Freeze the split, seal the diseased sheet. *done* Fit beat [152, 204], three held-out beats, 147 held-out frames, disjoint by construction and asserted. **17,499 scored nodes, and the evaluation mask is frozen from the recording alone** — never derived from a model setting, because the inherited mask was defined through the boundary width, so sweeping the boundary moved the model and the ruler together. The diseased sheet is sealed **by content across all three files that carry it**, and the seal was watched refusing.

6. And the premises — what the loop is allowed to call a tissue. **done** Written from the Okuda folder's experience, where a great deal had to be fixed before the loop could run at all. Eight claims, each one something a computer can *fail*. **Two of them do fail, and one of the two was only discovered because an outside audit noticed the check had never been written.**

- **The warm-up does not settle.** The fit runs a no-gradient warm-up “to the reproducible state” and then differentiates one beat. Give it one extra beat of warm-up and **the fitted window moves by 33%**. So the state the gradient is taken about is still drifting, and which beat gets fitted depends on how long the model was run beforehand. This premise was written down in Phase 1 and *not implemented* — the count read “six of eight” with nothing in the eighth slot — so it has been failing silently the whole time. It is the premise that underwrites every gradient in the project, and Phase 3 is called *certify the gradient*.
- **The model barely rests, and the tissue rests most of the time.** Over the same window the recording is quiescent **77%** of the beat — a sharp squeeze and a long relaxation — and the simulation **8%**. It contracts almost continuously. *And the objective cannot see this at all*, being invariant to timing by construction.

Both are graded *usual* rather than *certain*, so a run breaking them is **ambiguous**, not invalid — but neither is waived, and both go to the loop as things it must explain. **Eleven of thirteen checks hold.**

The eleven that do are worth stating, because they were assumptions until this phase: a resting sheet rests *exactly* (interior displacement is bit-zero with the muscle off); nothing leaves the dish and nothing goes non-finite; the solver sits **30 times inside** its own stability bound, now read from the run rather than from a comment; the seed reaches the engine's own generator; every operator in the fitting recipe actually runs; and the recorded beat is a beat.

And one more check, which needed a fit to test and so was tested on a fit built to fail it. A bounded parameter that ends *at* its bound has not found an optimum, it has found the edge of the box we drew — and the previous campaign drew conclusions about gain, duration and stiffness without ever asking. The check now refuses a fit pinned at 100% of its range and passes one in the middle, both watched.

The seal had to be attacked three times before it held, and that is the part worth reading. Version one could not read a compressed archive at all. Version two normalised for scale but still compared two million rounded numbers for exact equality, so a rescale by an arbitrary factor walked straight through. The third asks the right question — *is this the same measurement?* is a similarity question, not an equality one — and compares the shape of the motion over time. It now refuses the sealed specimen re-saved in four different unit conventions, cropped to a third of its area, and subsampled. The margin is not a guess: two registrations of the same specimen agree at **0.991–1.000** and the two different specimens at **0.151–0.224**, so the threshold sits in an empty gap, placed deliberately on the cautious side — letting the held-out specimen through is unrecoverable, refusing an innocent file is a nuisance.

Exit gate — met, and this phase is closed. The split exists with disjoint frame sets and the sealed signatures; the ceiling, the tracker number and the resolution table are in this note; the seal was watched refusing the held-out specimen through six different disguises; the premise checks run, and **what they refuse is recorded, not waived**; and Phase 1b is green.

Two things are on the record as broken rather than fixed, because fixing them is not Phase 1's job and hiding them would be worse than either: **the warm-up does not settle** (33% shift for one extra beat) and **the model rests for 8% of the beat where**

the tissue rests for 77%. The first is Phase 3's to answer, the second the loop's.

What Phase 1 cost. An afternoon, and it produced four numbers that bound everything afterwards — the ceiling, the tracker floor, the discretisation dependence of the enclosure axis, and the price of a regeneration — plus two defects in the apparatus and three in the checks I wrote to find them.

Phase 1b — The corpus we already own

done

Why this is worth its own list. Seventy-one MPM runs sit finished in `graphs_data/material` — trajectories, specs and movies, 45 GB. (They look lost because `log/material/` keeps only the recipe and a completion marker; the data is in `graphs_data/`.) Ten of them drive tissue with active traction of a pattern *we chose*: horizontal, vertical, swirl, radial, four quadrants, three travelling phase patterns, a central spot, and the cardio recipe itself.

That makes them the thing an instrument is supposed to be certified against — **loops whose ground truth is their own recipe** — at the real particle count, through the real forward model, for no compute at all. Until now the only known-answer cases available were ellipses drawn by hand.

And read as Track A rather than as instrument-testing, they already say something. Reading each run on its own axes — no recording involved — the orientation of the trajectories separates the horizontal driver from the vertical one (2.68 versus 1.57 radians, a right angle apart), which is the first entry in the map: *the axis you drive along is the axis the loops line up with*. Trivial, and that is the point — a map has to start from things that had better be true.

Already done, and two of them found something.

- **The descriptors read all ten**, and pass the cross-talk test on real output: rotate a 16,000-particle trajectory against itself and *only* the orientation axis moves, by exactly the angle applied, while magnitude, opening and direction hold at 1.0000. The first version failed that check — it used a peak that was not rotation-invariant, so turning a loop on the spot changed its “magnitude”. Caught by the check, not by inspection.
- **The coordination blindness is now demonstrated on the real forward model, not a toy.** Give every particle an independent random timing offset — destroy the synchrony completely — and the objective the previous campaign ranked on returns **exactly 1.0000, on all ten runs**, identical to a perfect match. Meanwhile its zero-motion null ranges from **+0.033 to +0.124** depending on the run, so even the origin is not a constant.
- **The visual instrument is rebuilt.** Both inherited picture-makers crash today on a module deleted with a sibling directory, so the campaign's primary figure could not be drawn at all. The replacement depends on nothing outside this folder, and it draws each run's beat beside the same beat with its coordination destroyed — two rows that no number in the inherited apparatus can tell apart.
- **Seven of the ten specs no longer run**, and the cause is exactly the defect that destroyed six batches: `pulse_stimulus` and `phase_delay_pulse` were *merged* into `activation_pulse`. The trajectories remain valid evidence — they are outputs — but the corpus is not reproducible until the specs are migrated.

Still to do.

- **Regenerate the important runs, starting with the broken ones** — `reproduce.py`. All eight stale recipes migrate and load, and the first has been regenerated and compared. **The answer is no: the merge was not behaviour-preserving.**

The comparison had to be done carefully, because the obvious version of it is misleading.

The regenerated trajectory differs from the archived one by 1.5×10^{-4} after three frames — about two hundredths of a grid cell, and 0.7% of the motion in the full run — and it grows monotonically from the very first frame (2.8×10^{-5} , 6.6×10^{-5} , 1.5×10^{-4}). Small, but is it the merge or is it the hardware? The discriminating experiment is cheap: run the migrated recipe *twice*. **Two runs of it are bit-identical** — exactly zero — so none of the divergence is hardware and all of it is attributable to the change in the library.

Two things follow, and the second is the more useful. *The merge changed the forward model*, quietly, and every archived trajectory in the corpus was produced by a version of the code that no longer exists — so the archive is evidence about a model we can no longer run, and the migrated recipes are a slightly different model that we can. *And the generate path is bit-deterministic*, unlike the fitting path, so once a recipe is migrated its runs are exactly reproducible.

Everything was written to a scratch root — the archive was never the thing being overwritten. Cost, measured: 137s of start-up and 24s per frame, so the 250-frame recipes are about two hours each. That is a fact Phase 2 needs before it budgets anything, and it is why the merge question was settled on three frames rather than two hundred and fifty: identical operators agree at frame one, and different ones separate at frame one, which is exactly what happened.

While writing it: the library kept backward-compatible aliases for three of the six re-names (`pulse_to_active_stress`, `p2g`, `g2p`) and not for the other three (`mpm_drag`, `pulse_stimulus`, `phase_delay_pulse`). There is no rule — which is how the same defect keeps recurring.

- **Write down what each recipe predicts, before reading the measurement**, and see whether the descriptors reproduce it: swirl should enclose more than a fixed axis; horizontal and vertical should differ *only* in orientation; radial should barely enclose anything. The predictions go on the record first.
- **Use the four phase-delayed runs as the coordination test set.** They were built to carry travelling-wave structure, so they are the natural benchmark for any measure claiming to see coordination — including the one Phase 2 has to invent.
- **Render the movies beside the loops**, so the eye and the number are looking at the same specimen.

Phase 2 — Deciding what may be believed about a number done CLOSED STOP

Goal. Give the loop numbers it can trust. *Not new numbers* — the measurements this campaign needs were written over three weeks in the old folder and audited in July. What was never done is deciding what may be believed about any of them.

The fault that started it, and it is one fault four times over. The same quantity acquired **four names, in four places, on four different sets of nodes**, and nothing recorded which was which: `loopscore_residual`, `enclosure_row`, `morphology_row` (withdrawn in July and still printed by every run) and `descriptors.py` — the last of which is mine, written one turn after being told to measure the loops, which is the same mistake again. So Phase 2 invents no ruler. It names the ones that exist and puts each through the same five tests.

THE APPROACH: five tests, and they answer different questions. No one of them is sufficient, and each was added because the ones before it had missed something.

	the question it answers	what it cannot see
1. the registry <code>metrics.py --check</code>	Is there one name, one definition and one implementation per quantity? Does a withdrawn metric refuse to compute? Does every declared zero say where it came from?	Anything about the numbers themselves
2. the battery <code>metrics.py --certify</code>	Distort the recording one axis at a time. Does it move on the axis it declares and hold still on the other eight?	Whether the number is <i>right</i> . And any change that is a symmetry of the population — see the openness defect below
3. the null bank <code>floors.py --nulls</code>	What do six models that know nothing score? Predict nothing, slide the sheet, copy the previous beat, interpolate from the edge, muscle off, fields untrained.	Whether a difference above that zero is larger than the noise
4. the floors <code>noise.py</code>	How much does it wobble when nothing changed? Beat against beat, the same seed twice, and seed to seed.	Whether the number means what its name says
5. calibration <code>calibrate.py</code>	Put in an ellipse, whose area, perimeter, reach, long axis and circulation all have a closed form. Is the number right?	Whether real tissue behaves like an ellipse

And then one rule, declared before the numbers arrived. Precision alone decides nothing: the three most precise quantities in the registry were, until this phase, the three that could support no claim at all, because they had no zero to be precise *relative to*. So

$$levels = \frac{|\text{what the tissue scores against itself} - \text{what knowing nothing scores}|}{3 \times \text{the largest measured noise floor}}$$

and a metric needs **five** of those steps before it may carry a claim — four to rank runs into quartiles, one spare. Below it, a metric can say *knows nothing* or *tissue-like* and has no opinion in between.

A role, which is not a tier. A tier says how much is known about a measurement; a role says what it is *for*. The quantity a fit descends and the quantity a claim cites are different jobs and there is no reason one number is good at both. `loopscore` spans 0.640 between its zero and the recording's agreement with itself, with a spread of 0.129 — **1.6 steps across its entire range**, which is the resolution at which the previous campaign ranked 324 runs and settled orderings on differences of 0.003. It is not defective, so it cannot be withdrawn, and it is the only number comparable with those runs and with the replay bar, so it is not deleted. It is marked *objective*: still optimised, still reported, and `cite()` raises if anything asks it to support a claim.

WHERE THAT LEAVES US. Seven instruments hold a zero, the battery and the beat-to-beat floor:

	zero	the tissue	steps	still missing
coordination	0.078	0.997	120.5	the fitted floor
path_length	0.0042	0.0001	39.5	the fitted floor
openness	0.339	0.006	36.6	the fitted floor; <i>and see below</i>
peak_excursion	0.0011	0.0000	30.2	the fitted floor
interior_r2	−0.831	0.899	12.1	the fitted floor
chirality_match	0.500	0.946	10.6	the fitted floor
orientation_error	0.785	0.058	10.5	the fitted floor
loopscore	0.070	0.710	1.6	<i>nothing — it is the objective</i>
loopscore_sd, $5 \times \text{residual}/$	—	—	—	a measured zero

Registry 17 checks of 17, battery 0 disagreements in 112 tested cells (14 more skipped as outside a declared domain), calibration 6 metrics against their closed form. **The one thing outstanding is the fitted-noise floor** — the same seed twice and several seeds, which need fits and are running. Every *steps* column above can only fall when they land, because the working unit is the largest floor and not the cheapest.

WHAT THE FIVE TESTS ACTUALLY FOUND. Each was added after the previous one let something through, and this is the list, because it is the argument for having five:

the grid	The reading surface's margin was ten nodes and the pinned band is wider: 36 of the 100 panels sat on particles tied to the answer, scoring a perfect 1.000 and lifting the picture by +0.17. Found in the July audit, never fixed, read on every dashboard since. Corrected to twenty. <i>My own check was wrong first</i> — it said 0 of 100 against the audit's 36, because the grid maps onto a sheet occupying 0.70 of the world and the factor was missing.
the null bank	Copying the previous beat scores +0.851, and the previous campaign's best fit was 0.545. The bar is a model with no physics in it. Also: the null bank was recorded in a vocabulary the registry does not contain, so nine of fourteen metrics had no measured zero and the two that did held hand-copied digits — <code>interior_r2</code> was typed -0.875 against a measured -0.8308 .
the null bank,again	Two metrics scored their best in the whole bank on the model that does nothing . <code>coordination</code> read a perfect 1.0000: a dead signal has no peak, so every lag comes back zero and they agree perfectly because they are all the same nothing. <code>residual/shape_detail</code> read +0.3031, because that family measures <i>recoverability</i> and everything is recoverable from nothing. Both now refuse. The battery could never have found either — every distortion starts from a real loop.
the pairing	<code>openness</code> , <code>path_length</code> and <code>peak_excursion</code> all took (model, recording) and used only the model . They described the model and never compared it, and a description has no right answer, so it has no score for knowing nothing, so its precision divides into nothing. Read on both sides and subtracted, they went from uninterpretable to the sharpest instruments here after coordination.
calibration	<code>openness</code> does not read how fat a loop is — on ellipses it returns $\pi/4$ for every aspect ratio, a circle and a needle alike. And it responds to orientation by 25% , which it does not declare, because its normaliser is an axis-aligned box that grows as the loop turns. <i>The battery cannot see this</i> : it turns every loop by one angle, and a population already pointing every which way is a symmetry of that. A coherent tissue is the case that matters and the case the battery never builds. Also: <code>path_length</code> was dropping the closing segment of a closed loop, so rolling a beat moved which segment went missing and it appeared to respond to <i>timing</i> ; and <code>orientation_error</code> is exact from 0 to $\pi/2$ but returns 1.4360 rad for two circles instead of refusing.

The openness finding is the one still open. Normalising by the loop's own axes, or using $4\pi A/P^2$, removes the orientation dependence and adds a genuine response to aspect — and invalidates every number measured with the current definition. That is a decision, not a patch, so it is stated and not taken. Until it is, a claim from **openness** must travel with `orientation_error`, which is separately measured and does read the axis.

What each measurement responds to, and how finely — drawn. Every entry in the registry is a claim of the form *this moves when X changes and ignores everything else*. Figure 1 is that claim as a picture: nine single changes to a loop across the top, the eight readable measurements down the side, and a cell saying whether the measurement moved or held.

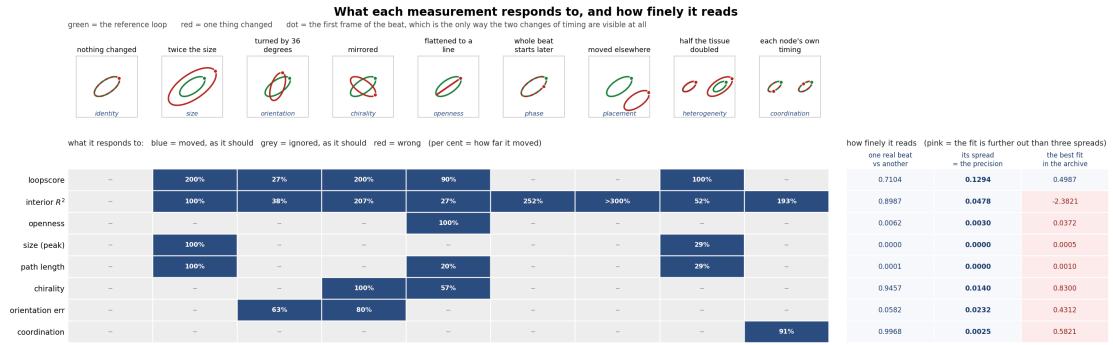


Figure 1. Left: the battery. Nine changes, each made one at a time, and no cell is red. The two changes of *timing* are invisible in any picture of a shape: only the dot marking the first frame has moved, and only two rows respond there. **Right:** the precision — what the measurement reads comparing one real beat with another, its spread, and the best fit the previous campaign produced. **Pink is the point of the figure.** Seven of the eight put that fit more than three spreads outside the tissue's own variation, and the one that cannot is **loopscore**, the composite the whole campaign ranked on: 0.4987 against a beat-to-beat 0.7104 ± 0.1294 . The specific measurements can, and they say where: the orientation is off by 0.431 rad where real beats vary by 0.058 , and the coordination reads 0.582 where real beats read 0.997 .

THE INSTRUMENTS ARE CLOSED. Seven measurements exist that a claim may rest on. Each has one name, one definition and one implementation; each moves on the axis it declares and holds on the other eight; each has a zero measured on six models that know nothing; each has the tissue's own beat-to-beat variation measured; and six of the seven have been read against a closed form and return the right number. Every defect found along the way is recorded on the class that carries it, with the measurement that found it. **That was the phase, and it is done.**

And the floor that was outstanding has landed. Every test named above has now been run, which is what closes this phase. What remains is one small measurement and one piece of work that belongs to a later phase:

the fitted floor

measured, and it did what was predicted of it. The seed-to-seed spread is **5 to 15 times** the tissue’s own beat-to-beat variation — coordination 0.0384 against 0.0025, openness 0.0340 against 0.0030 — and since the working unit is the largest floor, every *steps* figure fell:

	beat only	with seeds	
orientation_error	10.5	10.5	survives
chirality_match	10.6	8.7	survives
peak_excursion	30.2	8.6	survives
coordination	120.5	8.0	survives
path_length	39.5	6.5	survives
openness	36.6	3.3	RETIRED
interior_r2	12.1	2.3	RETIRED

It retired two instruments and promoted none, exactly as claimed — and **openness** is the one calibration had already caught reading $\pi/4$ for every aspect ratio, so two independent tests condemn it. **Five instruments survive.** Two caveats, both load-bearing. *The same-seed floor is still missing:* its run died of an out-of-memory error on a card another session had filled, so every ‘same seed’ cell read **nan** — and the promotion check reported *fits: yes* anyway, the same silence as the captioner, now fixed to check each of the three floors separately. *And depth is doing real work here:* at 300 iterations the fits have barely learned, with **orientation_error** landing at 0.770–0.796 against a null of $\pi/4 = 0.785$ and **chirality_match** at 0.569–0.606 against a null of 0.5. A spread measured across models that have not learned is not obviously the spread across models that have.

the same-seed floor

The one measurement still missing, and it is small: does the same command twice on the same card give the same answer? Its run died of an out-of-memory error on a card another session had filled, so every ‘same seed’ cell read **nan** — and the promotion check reported *fits: yes* anyway, the same silence the captioner had, now fixed to test each of the three floors separately. Re-running. It cannot raise a *steps* figure, only lower one.

the decomposition

The five **residual/** dimensions still have no measured zero. They are a decomposition of the objective rather than instruments in their own right, and they belong with Phase 2b, which is about fields.

THE STOP. At the end we know whether the fitted model beats *the best of the trivial alternatives* by more than the noise, with the comparison and the margin written into code before the numbers exist. **If it does not, the loop is forbidden to start.** This is a live outcome: copying the previous beat already outscores every fit the previous campaign produced.

Phase 2b — Fields: which ones the model needs, and which it would gain from IN PROGRESS (1 of 2)

Goal. The model learns four spatial fields — stiffness, gain, fibre orientation and prestress. Each is a chance to fit well for the wrong reason. Now that there are instruments with a measured floor, the question is finally answerable: *which of these does the tissue’s motion actually require?*

Two different questions, and only one of them is cheap.

removal	<code>ablate.py</code>	Take a <i>trained</i> model, neutralise one field, re-measure. Cheap — one forward pass each — and it is the right test for a field the model HAS.
addition	<code>add_field.py</code>	Train with the field and without it, same seeds, same depth. The only test for a field the model does NOT have: the others were fitted around its absence, so there is nothing to remove and nothing to conclude. Costs a fit per seed per arm.

A field’s null is a uniform field, not zero. Setting stiffness to zero changes how hard the tissue is, and the score would then move because the material changed rather than because its *pattern* mattered. Each field is replaced by its own mean — magnitude held, structure removed — which is the question actually being asked: does the model need this to vary in space? Prestress is the exception; its neutral value really is the identity tensor.

1. Removal, on the best fit in the archive. **done** `p3_b49_s2_fs2`, differences from the model as fitted:

field flattened	loopscore	orient. err	chirality	coordination
<i>(none — as fitted)</i>	+0.4987	+0.4312	+0.8300	+0.5821
stiffness	−0.4294	+0.0909	−0.0500	−0.0248
gain	− 0.6349	+ 0.1766	− 0.2500	− 0.1196
fibre	−0.3732	+0.1342	−0.1600	+0.0942
<i>3 × the working unit</i>	<i>0.3883</i>	<i>0.0695</i>	<i>0.0513</i>	<i>0.1153</i>

Orientation error is an error, so positive is worse; the rest are scores, so negative is worse. Four things follow, and the last is the reason this phase exists.

Gain is the field doing the work. Flattening it costs the most on every single instrument. **Flattening stiffness collapses the model onto the do-nothing baseline.** It scores +0.069, and predicting no motion at all scores +0.070. A model with a uniform stiffness is worth exactly nothing, which puts a floor under how much of this fit is the stiffness pattern. **No field changes the size of the loops.** `peak_excursion` and `path_length` do not move for any ablation, at all. The learned fields are shaping the loops and are not sizing them — and the size was never the failure, so this is consistent and worth having in writing.

And the objective could not have told us this. `loopscore` cannot resolve the fibre ablation — −0.3732 against a three-unit threshold of 0.3883 — so ranking on it would have concluded that the fibre field does not matter. `orientation_error` (+0.1342 against 0.0695) and `chirality_match` (−0.1600 against 0.0513) both say clearly that it does. **This is the demotion of Phase 2 paying for itself on a real question**, and it is the first time in this project that an instrument has said something the objective could not.

2. Addition: does prestress buy anything? **IN PROGRESS** The checkpoint carries stiffness, gain and fibre and **no prestress**, so removal cannot price it. Four seeds with the field against the same four without, at the same depth, judged against the measured seed-to-seed spread — the comparison the previous campaign made hundreds of times with nothing to make it against. Running.

what it can say	Whether prestress changes any instrument by more than three times the optimiser lottery, <i>at 300 iterations</i> .
what it cannot	Anything about 2400 iterations. And nothing about whether a field the model needs is one we have thought of — absence of a gain from prestress is not evidence that the missing physics is elsewhere.

The movies. `grid_movie.py` draws the ten-by-ten grid as the beat happens, one `mp4` per ablation, in `log/cardio_mpm/ablation_p3_b49_s2_fs2/`. A still says whether the shape matches; only a moving dot says *when* each part of the loop was traced, which is the axis the objective was blind to for sixty batches and the one `coordination` was built for.

Phase 3 — Certify the gradient

IN PROGRESS (1 of 5) STOP

Goal. Differentiability is this loop’s advantage, and right now it is an assumption that is false as stated: asked for a gradient through a specification file the library returns *nothing*, because nearly every operator turns its settings into plain numbers the moment it is built. The old trainer worked only by reaching around the library.

And one correction from the Plexus design document, which may change every budget here. It is emphatic that a loss should be **local in space and time** — a subset of points over a short rollout — because the backward graph grows as points $\times$ steps, while the evidence constraining any one mechanism is usually confined to a small, transient region anyway. The inherited fit is the opposite: a whole beat, ten sub-steps a frame, all 16,384 points, roughly 530 solver steps per gradient. That is very likely why one fit costs five and a half hours. Whether a local loss recovers the same mechanism far more cheaply is a Phase-3 experiment, and *if it does, every compute estimate in this note falls*.

This is also a real decision for you, which is why it is its own phase. Either we make the gradient work *through the Plexus language* — in which case the loop can propose genuine changes of mechanism, and every other Plexus project inherits the fix; or we accept the old trainer’s private list of switches — in which case this starts sooner and the loop can never propose anything its author did not anticipate. **I recommend the first, scoped to the dozen operators this problem actually uses**, because the second re-creates the previous campaign’s ceiling by construction.

- Make the cardio operators carry their settings as adjustable quantities. The mechanism for this already exists in the library and is used by no core operator.
- Fix the operator that writes into a buffer in place — it breaks the gradient after the second frame — and the one that overwrites the learned stiffness every frame.
- **Check the wiring before anything else.** Of the fifteen settings the previous campaign treated as levers, only six are actually connected to the optimiser at all — the rest are ordinary numbers that no gradient ever reaches. **“This lever has no gradient” and “this lever has no effect” are completely different statements**, and every later verdict depends on not confusing them. The wiring check runs first, and an unwired setting fails the gate rather than being quietly labelled uninformative.
- **Prove the gradient is the gradient.** Compare it against the brute-force answer for every connected setting. *Which settings are allowed to fail is declared in advance, from reading the code*, and committed before the test runs; anything else that fails is a failure, not a label.
- **Classify every knob:** does it carry a gradient, is it blocked by construction, or is it a switch that can never be learned? Four are already known to gate their own branch — the

setting decides whether its own code runs, so it cannot be learned from zero. This table is the loop’s map of what it can and cannot ask.

- **Recover a planted answer.** Simulate from settings we chose, at the real length and rhythm and with realistic noise, and fit them back. Two known traps are planted deliberately, including one already in the trainer: two different knobs that multiply the same thing, so only their product can ever be recovered. *An instrument that cannot recover an answer we planted may not judge one we have not.*

FIRST RESULTS — and the second test failed.

Wiring. `done train.py --grad_check` runs one forward and backward and reports what came back, before clipping and before stepping. **36 of 36 learnable tensors receive a non-zero gradient**, prestress included — which mattered immediately, because the prestress experiment then running would otherwise have been answering *is this field plumbed in?* while reporting an answer to *does the tissue have prestress?* Worth recording: the residual gradients are an **order of magnitude the largest** in the model, 4.7×10^4 against stiffness at 5.2×10^3 . **Is the gradient the gradient? not started NO — and the cause is found.** A central difference through the trainer’s own forward, at three step sizes, because a ratio near 1 at one step size is not proof: a correct gradient *converges* to the finite difference as the step shrinks.

	analytic	h	$h/4$	$h/16$	
raw_dur	−73.3	0.819	0.725	0.939	agrees
raw_g	641.0	1.310	1.309	1.313	flat at 1.31
f_ang	1014.6	0.850	0.833	0.721	does not converge
f_amp	227.8	0.991	0.979	0.803	does not converge
f_wl	−43.7	0.853	1.143	1.173	does not converge
f_ph	105.1	1.657	1.731	1.345	does not converge

One of six converges. `raw_g` is the clearest: three step sizes, ratio 1.31 every time. A flat plateau away from 1 is not truncation error — it is a gradient that is simply wrong, and no step size will rescue it.

The cause, and it is one line of the trainer. The 50-frame warm-up runs under `no_grad` with every map `.detach()`ed. The state the graded beat starts from therefore *does* depend on the parameters, and that dependence is cut out of the graph. **The analytic gradient is a partial derivative holding the warm-up fixed; the loss’s true derivative includes it.** Tested directly by shrinking the warm-up:

warm-up	analytic	finite difference	ratio
50 frames (the default)	641.0	839.0	1.309
1 frame	539.4	572.7	1.062

Removing the warm-up removes most of the error. **Every fit this project has ever run has been descending a surrogate that differs from its own loss by about 30% on gain**, and nothing in sixty batches could have noticed, because nothing compared the two. *And this meets the design document’s advice from the other direction.* Phase 3 already records that a loss should be **local in space and time**, on cost grounds. A shorter warm-up is both cheaper *and* more faithful, which is the rare case where the cheap option is also the correct one. The residual 6% at one frame is not yet explained and is the next thing to chase.

A useful thing found on the way. The forward is **bit-identical** on repeat (365.65628051757812 every time), so the non-determinism that makes two identical fits diverge lives entirely in the **backward**, not in the simulation. That is a different and more tractable problem than it looked.

Exit gate. A gradient reaches every learnable setting through the ordinary specification path; **the analytic gradient converges to the brute-force one for every connected setting**; the classification table is complete with nothing left unconfirmed; the planted settings come back within their stated error bars and **both planted traps are caught**.

Cost. About a day, one GPU.

Phase 3b — The algebraic inverse: injecting the data instead of integrating past it IN PROGRESS STOP

Goal. Decide whether the per-cell parameters can be recovered by *solving* rather than by descending — and if so, whether the campaign still needs a gradient at all.

What segmentation changed. Until Phase 2b the material was a continuum with a pattern painted on it, and heterogeneity was carried by coordinate networks: three SIREN fields, 397,827 parameters between them, none of which is a property of anything. With cells, the same heterogeneity is 472 numbers per property. The count is not the interesting part — the *structure* is. A field parameterisation enters the forward model through a network and is nonlinear in its weights. **A per-cell constant does not.**

THE OBSERVATION. The fixed-corotated stress carried by a material point is

$$\boldsymbol{\sigma}_p = 2\mu_p(\mathbf{F}_p - \mathbf{R}_p)\mathbf{F}_p^\top + \lambda_p J_p(J_p - 1)\mathbf{I} + g_{c(p)} a(\mathbf{x}_p, t) \mathbf{n}_{c(p)} \mathbf{n}_{c(p)}^\top, \quad (1)$$

and the Lamé constants are *linear* in Young's modulus, $\mu = E/2(1+\nu)$, $\lambda = E\nu/(1+\nu)(1-2\nu)$ [entities._lame].

The motion this stress produces is the momentum balance,

$$\rho \ddot{\mathbf{x}} = \nabla \cdot \boldsymbol{\sigma} - k \dot{\mathbf{x}}, \quad (2)$$

with k the drag. Equations (1) and (2) are the model; everything below is bookkeeping on them.

FROM (1) TO (2): what $\nabla \cdot \boldsymbol{\sigma}$ IS, discretely. In MLS-MPM the divergence is never formed; it is what the scatter to the grid computes. Each particle contributes to the grid nodes g of its stencil, with B-spline weight w_{gp} ,

$$\mathbf{m}_g \mathbf{v}_g = \sum_p w_{gp} \left[m_p \mathbf{v}_p + \left(\underbrace{m_p \mathbf{C}_p}_{\text{affine, known}} - \frac{4\Delta t}{\Delta x^2} V_p \boldsymbol{\sigma}_p \right) (\mathbf{x}_g - \mathbf{x}_p) \right], \quad (3)$$

[mpm_scatter: stress = (-dt*4*inv_dx**2)*p_vol*stress; affine = stress + mass*C] and the new particle velocity is gathered back, $\mathbf{v}_p^+ = \sum_g w_{gp} \mathbf{v}_g$. Composing the two and keeping only what carries $\boldsymbol{\sigma}$ gives the discrete divergence explicitly as a *linear operator* $\mathcal{D}$ on the collection of particle stresses,

$$(\mathcal{D}\boldsymbol{\sigma})_p := -\frac{4}{\Delta x^2} \sum_g \frac{w_{gp}}{m_g} \sum_q w_{gq} V_q \boldsymbol{\sigma}_q (\mathbf{x}_g - \mathbf{x}_q), \quad (4)$$

so that the one-step particle acceleration is

$$\mathbf{a}_p = (\mathcal{D}\boldsymbol{\sigma})_p + \mathbf{r}_p, \quad \mathbf{r}_p := \underbrace{\text{affine } \mathbf{C} \text{ transport}}_{\text{from observed } \mathbf{v}} + \underbrace{\text{gravity}}_{\text{drag}} - \underbrace{k \mathbf{v}_p}_{\text{drag}} + \underbrace{\text{wall}}_{\text{boundary}}. \quad (5)$$

Every symbol in $\mathcal{D}$ and in $\mathbf{r}$ — the weights w_{gp} , the nodal masses m_g , the volumes V_p , the positions — is fixed by the configuration. If the configuration is *injected* from the recording rather than produced by integrating, they are all data. That is the whole content of the injection: it converts $\mathcal{D}$ from something that depends on the unknowns into a known matrix.

FROM (2) TO (9): moving the unknowns to one side. Factor (1) into the part each parameter multiplies. Writing $c(p)$ for the cell owning particle p ,

$$\begin{aligned}\boldsymbol{\sigma}_p &= E_{c(p)} \mathbf{S}_p + g_{c(p)} \mathbf{T}_p, & \mathbf{S}_p &:= \frac{1}{1+\nu} (\mathbf{F}_p - \mathbf{R}_p) \mathbf{F}_p^\top + \frac{\nu}{(1+\nu)(1-2\nu)} J_p (J_p - 1) \mathbf{I}, \\ & & \mathbf{T}_p &:= a(\mathbf{x}_p, t) \mathbf{n}_{c(p)} \mathbf{n}_{c(p)}^\top.\end{aligned}\tag{6}$$

$\mathbf{S}_p$ is the stress the particle would carry at unit stiffness and $\mathbf{T}_p$ at unit gain; both are computed from the observed $\mathbf{F}_p$ and the known activation. Substituting (6) into (5), and using that $\mathcal{D}$ is linear and that each particle belongs to *exactly one* cell, the sum over particles regroups into a sum over cells:

$$\mathbf{a}_p = \sum_{c=1}^C E_c (\mathcal{D}[\mathbf{S} \mathbf{1}_c])_p + \sum_{c=1}^C g_c (\mathcal{D}[\mathbf{T} \mathbf{1}_c])_p + \mathbf{r}_p,\tag{7}$$

where $\mathbf{1}_c$ keeps only cell c 's particles. (7) is (9): stack p and the frames down the rows, read the columns off directly,

$$\mathbf{A}_{:,E_c} = \mathcal{D}[\mathbf{S} \mathbf{1}_c], \quad \mathbf{A}_{:,g_c} = \mathcal{D}[\mathbf{T} \mathbf{1}_c], \quad \mathbf{b} = \mathbf{a}^{\text{obs}} - \mathbf{r},\tag{8}$$

and the passage from (2) to (9) is, as in the companion note, nothing but moving across the equals sign everything the recording already determines [linalg_note.tex, §1].

Two things (8) hands us for free. A column is one scatter–gather with a single cell's particles carrying stress, so $\mathbf{A}$ is assembled by $2C$ cheap passes rather than by differentiating anything. And a cell touches only the grid nodes near it, so $\mathcal{D}[\mathbf{S} \mathbf{1}_c]$ is **spatially localised**: $\mathbf{A}$ is sparse, $\mathbf{G}$ is banded up to the range of the stencil, and two cells far apart are exactly orthogonal in the fit. The coupling that can hide a degeneracy is between *neighbours*, which is where to look.

Collecting (8),

$$\mathbf{A} \boldsymbol{\theta} = \mathbf{b}, \quad \boldsymbol{\theta} = (E_1, \dots, E_C, g_1, \dots, g_C) \in \mathbb{R}^{2C}, \quad C = 472,\tag{9}$$

with $\mathbf{b}$ carrying the drag, the boundary and anything else independent of $\boldsymbol{\theta}$. The least-squares solution obeys the normal equations

$$\underbrace{\mathbf{A}^\top \mathbf{A}}_{\mathbf{G}} \hat{\boldsymbol{\theta}} = \mathbf{A}^\top \mathbf{b},\tag{10}$$

and $\mathbf{G}$, the Gram matrix, is also the Hessian of the cost — for this problem exactly, not merely to Gauss–Newton order.

Three consequences, and each is worth more than a faster optimiser.

The fibre angle is the only nonlinear parameter. $\mathbf{nn}^\top$ is nonlinear in the angle, but it is one scalar per cell, and the active tensor can be solved for as a symmetric matrix and its axis read off afterwards. So the problem is 944 linear unknowns plus 472 one-dimensional ones, not 1,416 coupled nonlinear ones.

Non-identifiability becomes computable rather than discoverable. A degenerate direction is $\boldsymbol{\eta} \neq \mathbf{0}$ with $\mathbf{A} \boldsymbol{\eta} = \mathbf{0}$; then $\mathbf{A}(\boldsymbol{\theta} + \lambda \boldsymbol{\eta}) = \mathbf{b}$ for every λ , and no amount of fitting can

separate them [linalg_note.tex, §2]. Here that has a physical reading the Flyvis case does not: a degenerate direction is a set of per-cell stiffnesses and gains that produce the *same tissue motion* — a stiffer cell beside a softer one against the reverse. **We do not have to run a campaign to find out which cells are determined; it is the spectrum of $\mathbf{G}$, and the badly-determined ones can be drawn as a map.**

And it does not need the gradient that Phase 3 found to be wrong. There is no rollout, so there is no backward pass through 530 solver steps, so the detached warm-up — which makes the analytic gradient a partial derivative differing from the true one by about 30% on gain — does not arise at all.

WHAT MUST BE TRUE, AND IS BEING MEASURED.

superposition	(9) is only valid if the response to two disjoint sets of cells is the sum of the responses. The scatter operator takes a_max (the cardio spec sets 200) and the grid update clamps at the wall — a clamp is not linear , and whether it bites at these settings is a measurement, not an assumption.
the identity holds	$\ \mathbf{A}\boldsymbol{\theta}_{\text{true}} - \mathbf{b}\ /\ \mathbf{b}\ $ at the parameters the simulation actually used. If that is not small the formulation is wrong, and which of the two it is matters more than the number.
recovery	plant per-cell values, simulate, inject, solve, compare. <i>An instrument that cannot recover an answer we planted may not judge one we have not.</i>
noise	the method differentiates the observed positions, and the observation is PIV. Twice differencing amplifies error brutally. The level at which recovery stops being useful is the number that decides whether this works on real data at all — and the weak form (multiply by test functions, integrate by parts) removes the second derivative if it does not.
scaling	$E \sim 100$ and $g \sim 1$, so an unscaled $\mathbf{A}$ will look ill-conditioned for a trivial reason. cond must be reported on a column-normalised $\mathbf{A}$ or it is an artefact of units.

The honest limit of the method. This is *equation* error, not *output* error: it asks which parameters satisfy the balance frame by frame, not which reproduce the trajectory when rolled out. The two differ, and equation error is the biased one under measurement noise. So the structure is not a replacement but a division of labour — **solve for the parameters algebraically, then judge them by rolling the model out with the border anchored to the recording, exactly as the prototype already does.** The anchor stays.

And it does not replace the loop. Solving (10) estimates parameters *within* a fixed mechanism. If the model has no prestress, no per-cell stiffness repairs that. What changes is the quality of the evidence the loop receives: the residual after an algebraic fit is attributable to **individual cells**, not smeared across a smooth field, which is a sharper input to mechanism discovery than anything the campaign has had.

THE STOP. If superposition fails, or the planted parameters do not come back at zero noise, the algebraic route is closed and Phase 3's gradient is the only way through — and that must be said rather than patched around. If it succeeds, the honest consequence is larger than a speed-up: **most of what the agentic loop was going to spend its GPU-hours on is a linear solve**, and the loop should be re-scoped onto the question it is actually for.

Phase 4 — What a fit may claim, and the loop that enforces it **NOT STARTED** **STOP**

Goal. Turn “this lever does nothing” from a shrug into a measurement, decide how big the model is honestly allowed to be, and then build the four steps so that the answers are enforced rather than remembered.

- **Replace “structural cap” with three verdicts.** Every flat response returns exactly one of: *smaller than the noise*; *the recording cannot resolve it*; or *the response really is flat*. Only the third deserves to be called a limit. One backward pass replaces a five-point sweep, and *the compute that saves is itself a Track-A result worth recording*.
- **Name the degeneracies in English.** Which combinations of settings the recording cannot tell apart — “these two knobs are one knob” — computed both with the learned fields held still and with them free, because those are two different questions and reporting one alone is a new way to manufacture a law.
- **How big may the model be?** Fit at increasing sizes and read the held-out score. The honest size is the smallest one whose held-out score is within one noise unit of the best. It is then frozen, and the gate enforces it.
- **Are the learned fields a material property or a sponge?** Two tests, both cheap. Take the stiffness and fibre maps from the fit beat and apply them *unchanged* to a held-out beat — a material property transfers, a per-beat residual sponge does not. And compare the learned fibre directions against the fibre directions visible in the raw microscope image, which is on disk and has never been opened for this purpose. **That is the only chance in this project to give a learned field a physical referent from outside the fit.** Both thresholds are written down before the tests run.
- **Then build the four steps — and attack them.** Feed the loop a proposal that cannot fail, and watch the gate refuse it. Feed it a prediction naming an uncertified quantity, and watch it come back *inconclusive* rather than confirmed. Feed it a deliberately wrong prediction and watch the arithmetic mark it refuted. Feed it a half-converged fit, an effect below the noise, a mechanism the recording cannot see, and a model bigger than the honest size — and watch each refusal fire.

THE STOP. If the held-out score at *every* model size sits inside the noise band from Phase 2, then there is no mechanism to discover with this apparatus on this recording, and the loop does not start. Said out loud now, before the numbers arrive.

Exit gate. The three-way classifier returns the right verdict on three constructed cases, one of each kind; every refusal above actually fires and is recorded with its reason; the loop’s default score is the held-out one, with a test proving an in-sample number cannot be registered as a prediction target; and the roster document and the code agree in both directions.

Cost. About two days on two cards.

Phase 5 — The first live round **NOT STARTED**

Goal. One round, attended, on the healthy sheet — and it is aimed at questions **whose answers Phase 2 already wrote down**. If the loop lands somewhere else, the loop is broken, not the physics.

What you get. The first honest entry in an empty register, and a list of everything the round revealed about itself. The Okuda folder’s first live round produced one usable run out of eight and seven outputs whose recipient turned out to be nobody; that was money well

spent, and this one should be read the same way.

Phase 6 — The campaign, and the map

NOT STARTED

Goal. Run unattended. The product is a *map*: which mechanism moves which part of the error, with an error bar on every arrow, and which parts of the error nothing we have can move. Every row carries its held-out effect, its gradient, its identifiability verdict, and its capacity-matched control. **A dose-confirmed nothing is published as a result.**

The number that says it is working is how often the loop was *surprised* — how often a written prediction turned out wrong. A campaign that is never surprised has bought coverage and learned nothing.

Phase 7 — Breaking the seal: healthy, then diseased

NOT STARTED

Goal. Fit the healthy sheet. **Predict the diseased one.** The claim we would like to make is that the same mechanisms, with a small and stated set of parameters changed, account for both — and that the loop found which parameters those were.

Opened once, and rehearsed first. The prediction is written, checksummed and time-stamped before the key to the sealed specimen exists, and it names a direction, not just a difference. The whole procedure is rehearsed end to end on a held-out *healthy* beat treated exactly as if it were sealed, so the one shot is not spent on a bug. **A refutation closes this phase exactly as a confirmation does**, and is written up with equal prominence — and because there is only one diseased specimen, “we could not tell” is a permitted third outcome, reported with the size of effect we would have been able to see.

What you get. *The figure*, and the only generalisation test this dataset can offer.

Phase 8 — The reckoning, and the method claim

NOT STARTED

Goal. Of the previous campaign’s claims, how many did we re-test and what happened to each; and what, in numbers rather than adjectives, did differentiability buy?

What you get. Three tables — the reckoning over every inherited hypothesis; the audit of our own register, every entry carrying its split, its error bar and its run folder; and the method claim as measurements: levers settled per GPU-hour, flat responses the gradient resolved that a sweep could not, the surprise rate, and the retraction rate. Alongside it the honest comparison: sixty batches, no error bars, nothing held out, no surviving claim — against whatever this campaign actually earns.

7. Phase 0 report — 2 August

*This is a phase boundary. The gate prints **PASS, 18 of 18**, the canaries catch **8 of 8**, and one thing is deliberately **not** certified and is recorded as a number rather than a footnote. I have stopped here.*

What now exists. A folder that runs: `ingest.py` (rebuild the recording), `data.py` (one path, no fallback), `determinism.py` (fix the dice), `provenance.py` (a run carries its own source), `train.py` (the repaired apparatus), `certify_apparatus.py` (the gate, with canaries), and the two registers.

The gate, in one line each.

what	what happened
it runs at all	The inherited trainer died on every invocation, on a function one branch had deleted from under another. Repaired; a short fit now completes end to end.
the recording rebuilds	It was not in the repository and the script that made it was gone. It is now rebuilt from the microscope derivatives by committed code and matches the existing file bit for bit , every array.
one path, no fallback	The loader used to try three locations and take whichever existed — a cure that turns “file missing” into “fitted the wrong recording”. Now one explicit path, and both refusals were watched firing.
the dice are fixed	Two fits at one seed are now bit-identical over 198,407 parameters ; two seeds differ. Neither was true of anything in the previous sixty rounds.
a run carries its source	Every run copies the bytes of every module it imported into its own folder, with hashes. A branch switch or a concurrent campaign in the same tree cannot reach backwards into a finished run — which is exactly how two batches were lost before.
the ledger is out of the code	Nine phrases from the retired ledger were compiled into comments, help strings and one default. The scan is clean, and the canary proves the scan still fires.
the registers exist	38 inherited claims, every one carrying an open status. The belief register has zero entries.

Three things found while doing it, none of them planned.

- **The default setting was a retired belief.** `--stiff_src` still defaulted to an image-sourced field whose implementation had been deleted, so the default invocation trained a two-parameter stub against a synthetic blob — and the dashboard’s “correlation with the microscope” panel correlated against that same blob. The path is removed *because the code is gone*, not because the claim about it was accepted; the claim is an open question like the rest.
- **A silent bug in the pulse.** The learnable duration was initialised against a hard-coded bound while the simulation used the one on the command line, so `--dur0 10 --dur_hi 11` actually started at 8.09. Every duration claim in the previous campaign was made through that.
- **My own first fix was wrong, and the check caught it.** Making the arithmetic deterministic meant replacing a sampling routine with an explicit equivalent. The first version had the two axes swapped — and disagreed with the original by *more than the size of the signal*. It was caught because the replacement was checked against the thing it replaced before being used, which is the rule this whole phase exists to install: *the instrument lies before the physics does*.

The one thing that is not certified, stated as a number. Determinism is complete on the processor and **not** on the graphics card. The blocker is named: `grid_sampler_2d_backward_cuda` has no deterministic implementation, and it is reached from inside the shared Plexus library — `Field.sample`, called by the active-stress operator every frame. Fixing that changes a library three other projects are using, so it is not Phase 0 work.

Rather than relax the gate, the spread was *measured*: three identical runs on the card agree to **3.0e-6**, a relative 7.5×10^{-8} , and only one of the three pairs was actually identical. That is small, it is not zero, and it will grow with the length of the optimisation — so Phase 2 re-measures it at the depth the loop will really use, where it becomes one of the two noise floors the campaign is built on. A run may only become irreproducible on purpose: the flag that permits it is off by default and is recorded in the run’s own manifest.

8. An outside audit, and what it found in here

The Okuda folder was heavily reworked on 2 August, and rather than guess at the consequences we had three independent auditors read both folders and answer one question: does any of it change

what is already closed here? *Every finding below was then re-verified by hand before anything was altered.*

The answer to the headline question is no, and the reason is worth keeping. Of roughly sixty commits over there, six touch the new reasoning machinery; the rest repair a live cluster runner with a vision model in it — a machine this project does not have and will not have until Phase 5. Porting it would mean building theirs, which is the opposite of the instruction this loop was given. More usefully, their own conclusion supports the shape chosen here. After nine rounds they found, across forty-four thousand words of agent-written reasoning, *not one* occurrence of “only tested once” or “cannot conclude”, against fifteen assertions of closure — and concluded: *the agents were exactly as rigorous as the structure they were given, and not one degree more. Logic is not an emergent property of putting capable models in a loop; it is a structure.* That is the argument for giving every deterministic question to code, which is what the five-agent, nine-check roster already does.

One piece specifically does *not* transfer. Their independence rule counts distinct combinations of switched-on operators. Ours would be seed, initialisation, beat and specimen — and we have one specimen. Lifted over, it would return “one independent observation” for everything we will ever measure.

What it did find, in here, was six defects — and none of the measured numbers is wrong.

what	what was actually the case
a check that did not exist	Premise 5 — <i>the warm-up must settle</i> — was written into the premise list and never implemented , while the verdict still printed “6 of 8 held” because a sub-check of premise 8 was sitting in the eighth slot. The denominator hid the gap and the phase read as complete. Now implemented, and a coverage check reads the headings out of the premise document and fails when a declared premise has no code behind it.
the record did not match the note	Both machine-readable records were whole-file overwrites, so the last and narrowest run won. The resolution file held three of seven rows — the table printed in this note was <i>not</i> backed by what was on disk. Both now merge, and all seven rows are restored from the surviving arrays.
a check reading its own comment	The stability premise hard-coded the grid size, the sub-step count and the bound as literals, so it computed the wrong number on the four ladder runs that varied exactly those. Its own text says the bound must be <i>derived, not left in a comment</i> — and it was reading the comment.
the instrument that reached nothing	descriptors.py — the measurement Track B is judged by, with a self-test, which produced every number in Phase 1 — was imported by three analysis scripts and by no fit . Every fit still wrote only the inherited, uncertified row. Found by applying their own dead-machinery test to our code, where it is word for word the defect they had just spent a day on.
a register that described machinery it did not have	The belief register announced “a hard length cap enforced by code” and a retraction file. Neither existed anywhere. Both are now real — a 120-line cap, the file on disk, seven required columns per row — and three new canaries , so the register check has now been watched refusing.
the recipe that misdescribed the run	Four operators were declared in the recipe and never stepped: the fit replaces them deliberately with a learnable pulse and its own material maps. The premise was right to refuse it. There is now a <i>fitting</i> recipe that declares what the fit runs, and removing them is proved to change nothing — fits under both are bit-identical over 198,407 parameters. A description was corrected, not a model.

Two things we had written down had to be said less strongly.

The ceiling. “No model may be scored above 0.705” was stated flatly, using an instrument this same phase had shown returns a perfect score for a tissue whose timing has been destroyed. A score above the ceiling has at least two readings and we had given one. Phase 2 now carries an

extra item: recompute the ceiling and the tracker floor with whatever ruler it certifies.

The tracker. The four beats agreeing looked like corroboration and was not. Strip the subject — the tracking software — and the same specimen, the same movie and the same pair of algorithms remain in all four. That is one observation repeated four times. The claim is strong for this specimen and untested on any other, and the second signature that would settle it sits inside the sealed specimen.

And two things a fit could not previously say about itself. Whether it finished — a crash, a wall-clock kill and a clean finish left the same files behind — and how many of its optimiser steps were silently discarded, which could have been most of them without any trace. Both are now written into every run, and the premise checker is importable so that nothing in Phase 2 can compute a number without being able to ask whether the run it came from was admissible. That last one is the whole point: their most expensive defect was a demotion that existed in code and had never once fired.

9. The lesson we are starting from, in three lines

Sixty rounds produced a confident ledger, and emptying it took three measurements: the random seed was never set, so identical runs differed by most of the signal; neither score was ever read against its own null, so nobody could say whether anything had been achieved; and nothing was ever held out, so “it fits” and “it is right” were never separated. **None of the three is about biology, and all three are cheap** — which is precisely why the pre-phases exist, and why the register starts empty.

10. What I need from you

Nothing blocking — Phase 0 can start today. Four things worth your decision, in order of how much they change the work:

1. **Gradients through the Plexus language, or through the old trainer’s switches?** (Phase 3.) The first is a few days more work, makes every other Plexus project differentiable, and is the only version in which the loop can propose a mechanism nobody anticipated. The second starts sooner and rebuilds the previous campaign’s ceiling on purpose. *I recommend the first.*
2. **Do you accept the two stops?** Phase 2 can conclude that the model does not beat doing nothing, and Phase 4 that no model size clears the noise. Both would end the project in its current form. They are only worth building if it is agreed *now* that reaching one is a result and not a failure to be argued around.
3. **Is “fit the healthy sheet, predict the diseased one” the demonstration?** It is the only real generalisation test in this dataset, and it is a strong claim. It is also a single sample, so it should be chosen deliberately now rather than defended afterwards.
4. **How much of the project may rest on maps inside the sheet?** If Phase 1 confirms that the fine spatial pattern is mostly the tracking software, then learned stiffness and fibre maps are not a publishable product here, and the loop should be aimed at whole-sheet mechanisms instead. I would rather find that out in Phase 1 than in Phase 7.
5. **Is more data available, and how hard would it be to ask?** One substrate stiffness, no drug, no change of pacing, and one specimen per condition. Everything above is designed to squeeze that honestly, but a single unperturbed recording can only ever be fitted, never really

challenged. *One extra condition would be worth more than anything on this page* — so it is worth knowing whether the Utrecht group has more, before we spend weeks proving how little this can support.

And tell me if this note is the right shape. It is cheap to change, and it is the thing that lets you steer.

Appendix — words I could not avoid

Only these five. Everything else above is ordinary English.

- **Operator** — one mechanism as a reusable building block: “the muscle pulls along its fibres”, “the tissue resists being stretched”. A model is a combination of these.
- **Differentiable** — the simulation can be asked not only “what happens?” but “which way should I nudge this setting to match the recording better?”. Answering that is what makes a fit possible, and it is also what makes all the extra checks necessary.
- **The residual** — the part of the real recording the model still gets wrong, broken into named pieces. In this loop it is both the score and the agenda.
- **Identifiable** — a mechanism is identifiable if *this particular recording* could tell whether it is present. A mechanism that is real but not identifiable looks exactly like one that does nothing.
- **Phase** — one of the stages above. Our agreed stopping points.

The working log will be `SESSION_LOG.md`; the open questions inherited from the previous campaign `HYPOTHESES.md`; what we actually believe `BELIEFS.md`; the physiological premises `PREMISES.md`; the roster `ROLES.md`. You should not need any of them.